

INSTITUTE CO{)ELINE

Introduction to AI and Tools

DAY 1 — PRACTICAL TASKS WORKBOOK

Session: 4 Hours | 12 Tasks | Individual & Pair Exercises

Complete each task using the AI tool of your choice. Write your prompts and observations in the spaces provided. Tasks alternate between general business scenarios and software development contexts.

Participant Name:	<u>Maiyada Albarwani</u>	Date:	<u>09/09/2026</u>
Job Title:	<u></u>	Group:	<u>Batch 3</u>

■ Time Block	Section	Tasks	Focus Area
0:00 – 0:40	A — AI Understanding	1–2	AI concepts, tools overview, responsible use
0:40 – 1:30	B — Prompt Engineering	3–5	CRTFC framework, prompt quality, advanced techniques
1:30 – 2:30	C — Communication & Docs	6–8	Emails, documentation, meeting summaries
2:30 – 3:20	D — Data & Code Tasks	9–10	Document analysis, code-related AI tasks
3:20 – 4:00	E — Workflow Design	11–12	Automation templates, personal AI workflow plan

■ Blue tasks = General business context

■■■ Purple tasks = Software development context

■ Mixed = Both perspectives combined

Prompt 1 — Initial: Write a Python function that processes a list of customer orders and calculates the total order value.

Prompt 2 — Add error handling: Improve the previous function by adding appropriate error handling for empty input, missing fields, invalid prices, and invalid quantities. Keep the implementation simple and readable.

Prompt 3 — Add logging and tests: Improve the previous version by adding appropriate Python logging for successful processing and errors. Then generate pytest unit tests covering the normal case, empty input, missing fields, invalid values, and boundary conditions.

Prompt 1 — Initial: Write a Python function that processes a list of customer orders and calculates the total order value.

Prompt 2 — Add error handling: Improve the previous function by adding appropriate error handling for empty input, missing fields, invalid prices, and invalid quantities. Keep the implementation simple and readable.

Prompt 3 — Add logging and tests: Improve the previous version by adding appropriate Python logging for successful processing and errors. Then generate pytest unit tests covering the normal case, empty input, missing fields, invalid values, and boundary conditions.

SECTION A | AI Concepts, Tools & Responsible Use ■ 0:00 – 0:40

Complete Tasks 1–2 in this section. Start from memory, then verify using your AI tool. The goal is to build a solid conceptual foundation before applying AI to real tasks.

Task 1 AI Knowledge Audit & Tool Exploration

■ 20 min

● Beginner

■ Instructions:

1. Without using AI, write your own definition of AI and list what you think it can and cannot do (use the table below).
2. Now open ChatGPT or Claude. Ask: 'Explain what a Large Language Model is in simple terms for a software developer who has never used AI tools before.'
3. Compare the AI's explanation to yours. Note 1 concept that was new or surprising to you.
4. Try a second tool (e.g., if you used ChatGPT, try Claude or Gemini with the exact same prompt). Note any differences in the responses.
5. Ask one of the tools: 'What are the top 5 use cases for AI tools in software development?' Note its answer in the space provided.

■ **Deliverable:** Completed tables + 2 tool comparison notes + developer use cases listed.

My Definition of AI (before AI tool)

AI's Explanation (from the tool)

A tool that can search through a large number of resources fast.

Technology that enables computers to perform tasks that normally require human intelligence.

■ AI CAN (your view)

1. Do documentation work
2. Give information from good resources
3. Explain concepts and solve problems
4. Analyze data

■ ■ AI CANNOT / Risks (your view)

1. Give moral judgement on something
2. Give opinions on critical matters
3. Can Hallucinate
4. Doesnt have memory between sessions.

■ *Difference I noticed between Tool 1 and Tool 2:*
Write your answer / prompt here:

One tool gave a very basic and simplified answer with examples. The second tool was more like an academic style answer with very solid reasoning.

■ ■ ■ *Top 5 AI use cases for software development (from AI's answer):*
Write your answer / prompt here:

AI can act like a coding assistant that helps developers write, understand, test, debug, and improve code faster.

Task 2 Responsible AI — Risk Scenarios for Developers

■ 20 min

● Beginner

■ Instructions:

1. Read each scenario. Mark SAFE ■ or RISKY ■■ and write a one-sentence reason.
2. For the RISKY ones, write an alternative approach that achieves the same goal safely.
3. After completing your assessment, ask ChatGPT: 'What are the top 5 AI data privacy risks for software developers?' Compare its answer to your assessment.

■ **Deliverable:** Completed risk table + alternative safe approaches written.

Scenario	Safe / Risky?	Reason / Safe Alternative
You paste a function from your company's payment processing module to get a code review	Risky	Remove sensitive/proprietary details or use a sanitized example.
You ask AI: 'Explain how JWT token refresh logic works' without pasting any code	Safe	No company or personal data is shared.
You paste your production server's nginx config file including the server IP and domain names	Risky	Remove IPs, domains, credentials, and other identifying information.
You describe a bug scenario: 'I have a function that processes user objects — it crashes when email is null'	Safe	No sensitive data
You upload a CSV of customer order history (with names, emails, and order amounts) to ask for analysis	Risky	Anonymize or generate synthetic data first.
You ask: 'Write a Python script to parse CSV files with columns: order_id, amount, status'	Safe	No sensitive information is required
You paste your AWS .env file to ask AI why your connection string isn't working	Risky	Replace secrets with placeholders such as YOUR_PASSWORD_HERE.

You ask AI to suggest database index strategies for a table with 10 million rows	Safe	General technical question and doesn't require sharing sensitive or confidential information.
--	------	---

SECTION B | Prompt Engineering Practice ■ 0:40 – 1:30

Tasks 3–5 build your prompt writing skills from basic to advanced. Use the CRTFC framework. Test every prompt in your AI tool and refine based on results.

Task 3 Spot and Fix Weak Prompts

■ 15 min

● Beginner

■ Instructions:

1. Each prompt below is weak. Identify WHY it is weak, then rewrite it using the CRTFC framework.
2. Test your improved version in the AI tool. Rate whether it gave a significantly better result.

■ **Deliverable:** 3 improved prompts written + result quality ratings.

■ Weak Prompt 1 (General): "Write documentation"

Why is it weak? _____

CONTEXT	I am a software engineer working on documenting monthly performance
ROLE	Act like an experienced data analyzer working in a tech company.
TASK	Write a detailed documentation of the data given for performance
FORMAT	Make sure to include tables and graphs and make it less than 5 pages long.
CONSTRAINTS	Do not write any opinions

Result quality: ■ Much better ■ Slightly better ■ No improvement

■ ■ ■ Weak Prompt 2 (Dev): "Help me with my code"

Why is it weak? It doesn't have any of the CRTFC elements.

CONTEXT	I am a software engineer working on building application building school application
ROLE	Act like a software engineer working on a similar project
TASK	Look at this line of code and give me guidance or review about it
FORMAT	Return it in the same format
CONSTRAINTS	Make it short and explain

Result quality: ■ Much better ■ Slightly better ■ No improvement

■ Weak Prompt 3 (General): "Make a summary"

Why is it weak? It is very generic. No details.

CONTEXT	I am a student working on a research about benefits of coffee
---------	---

ROLE	<i>Act as an academic</i>
TASK	<i>give me summary of this research paper about coffee dependency</i>
FORMAT	<i>Make it in an academic report style</i>
CONSTRAINTS	<i>Do not include something that is ont in the research</i>

Result quality: ■ Much better ■ Slightly better ■ No improvement

■■■ Weak Prompt 4 (Dev): "Fix the bug"

Why is it weak? *Very general*

CONTEXT	<i>I am a software developer working on a hospital application</i>
ROLE	<i>Act as an experience developer</i>
TASK	<i>I faced a problem in my code and not sure where the problem is, help me fix the issue</i>
FORMAT	<i>Give me the code back in the same format</i>
CONSTRAINTS	<i>Do not add unnecessary things</i>

Result quality: ■ Much better ■ Slightly better ■ No improvement

■ Weak Prompt 5 (General): "Write an email about the meeting"

Why is it weak? _____

CONTEXT	<i>I am a web developer trying to schedule a meeting with the client</i>
ROLE	<i>Act as a developer</i>
TASK	<i>write me a short email requiring an urgent meeting</i>
FORMAT	<i>Make it formal.</i>
CONSTRAINTS	<i>Do not use a high tone</i>

Result quality: ■ Much better ■ Slightly better ■ No improvement

Task 4 Build a Level-4 Expert Prompt

■ 20 min

● Intermediate

■ Instructions:

1. Choose ONE scenario below (circle your choice).
2. Scenario A (General): You are a project manager. Write a status email to the steering committee summarizing milestone progress, a risk, and the next sprint goal.
3. Scenario B (Dev): You are a backend developer. Ask AI to review a Python function that processes payment transactions for correctness, security, and edge cases.
4. Scenario C (Dev): You are a tech lead. Ask AI to compare three caching strategies (Redis, Memcached, in-memory) for a high-traffic API with specific tradeoffs.
5. Fill in the CRTFC table below, then write your full prompt in a single paragraph.
6. Test the prompt. Rate the result. Then write one follow-up prompt to refine the output.

■ Deliverable: Full CRTFC prompt + follow-up refinement prompt + quality rating.

I chose Scenario: B

CONTEXT	<i>I am a backend developer working on a Python application that processes payment transactions. I need to ensure the function is reliable, secure, and robust before it is used in production.</i>
ROLE	<i>Act as a senior Python backend engineer and application security reviewer with expertise in payment systems.</i>
TASK	<i>Review the provided Python payment-transaction function for correctness, security vulnerabilities, error handling, edge cases, and potential performance issues. Identify problems and recommend specific improvements.</i>
FORMAT	<i>Use clear explanations and include code where appropriate.</i>
CONSTRAINTS	<i>prioritize issues that could cause incorrect payment processing or financial loss.</i>

■ **My Full Prompt** (combine all CRTFC elements):

Write your answer / prompt here:

I am a backend developer working on a Python application that processes payment transactions, and I need to ensure the transaction-processing function is reliable, secure, and robust before production use. Act as a senior Python backend engineer and application security reviewer with expertise in payment systems. Review the provided Python payment-transaction function for correctness, security vulnerabilities, error handling, edge cases, and potential performance issues. Provide the response in these sections: overall assessment, correctness issues, security issues, edge cases, error-handling issues, performance concerns, recommended improvements, and an improved version of the function. Prioritize issues that could cause incorrect payment processing or financial loss, including duplicate transactions, invalid inputs, missing values, failed transactions, and unexpected errors. Follow Python best practices and PEP 8. Do not assume external libraries or infrastructure unless explicitly stated. Clearly distinguish confirmed problems from suggestions, and do not expose or invent sensitive payment information.

■ **Follow-up refinement prompt** (what did you ask next to improve the output?):

Write your answer / prompt here:

Review your previous response again as a payment-systems security specialist. Prioritize the three most serious risks and explain why they are important. Then revise the proposed function so that it handles invalid input, duplicate transactions, transaction failures, and unexpected exceptions safely. Finally, provide 5 concise test cases that should be run before this code is considered production-ready.

Initial result quality:

■ Excellent ■ Good ■ Needs work

After refinement:

■ Excellent ■ Good ■ Needs work

Task 5 Advanced Prompting Techniques

■ 15 min

● Advanced

■ Instructions:

1. Practice each advanced technique using the instructions below.
2. **TECHNIQUE A** — Chain of Thought: Ask AI: 'Explain step by step how a JWT token is validated on an API server. Then identify 3 potential security vulnerabilities in a basic implementation.' Paste the response summary below.
3. **TECHNIQUE B** — Few-Shot Prompting: Write a prompt that gives the AI 2 examples of a code comment format you want, then asks it to apply that format to a new function.
4. **TECHNIQUE C** — Iterative Refinement: Start with a simple prompt, then write 2 follow-up prompts that progressively refine the output (e.g., add error handling, then add logging, then add a unit test).

■ Deliverable: 3 technique examples attempted + observations noted.**Technique A — Chain of Thought**

■ Write your prompt(s) here:

Explain the process of JWT token validation on an API server in a clear, step-by-step manner. Then identify three potential security vulnerabilities that could occur in a basic JWT implementation and explain how each vulnerability could be mitigated.

Technique B — Few-Shot Prompting

■ ■ ■ Write your prompt(s) here:

Use the following two examples to determine the code-comment style I want.

Example 1:

Validate the user's authentication token before processing the request.

Example 2:

Calculate the order total after applying the applicable discount.

Now write a single concise comment for the following function:

```
def process_payment(transaction):
```

Follow the same style as the examples. The comment should describe the function's main purpose, begin with an imperative verb, and contain no unnecessary detail.

Technique C — Iterative Refinement (show all 3 prompts)

■ Write your prompt(s) here:

Prompt 1 — Initial:

Write a Python function that processes a list of customer orders and calculates the total order value.

Prompt 2 — Add error handling:

Improve the previous function by adding appropriate error handling for empty input, missing fields, invalid prices, and invalid quantities. Keep the implementation simple and readable.

Prompt 3 — Add logging and tests:

Improve the previous version by adding appropriate Python logging for successful processing and errors. Then generate pytest unit tests covering the normal case, empty input, missing fields, invalid values, and boundary conditions.

SECTION C | AI for Communication & Documentation ■ 1:30 – 2:30

Tasks 6–8 apply AI to real communication and documentation scenarios. All prompts should use the CRTFC framework. Save your best prompts — you will reuse them in Section E.

Task 6 Professional Email Writing — General & Developer Scenarios

■ 20 min

● Intermediate

■ Instructions:

1. Write and test a professional email for BOTH scenarios below (one general, one developer-specific).
2. SCENARIO 1 (General): You are a product manager. Write an email to senior management requesting approval for a 2-week delay in the product launch due to critical QA findings. Keep it under 200 words, professional, and include a revised timeline.
3. SCENARIO 2 (Dev): You are a backend developer. Write an email to the DevOps team requesting emergency access to production logs to investigate a P1 incident impacting 15% of users. Include: the affected service, symptoms, and what you need access to.
4. For each: fill in the CRTFC table, write the full prompt, test it, and rate the result.
5. BONUS: Ask AI to rewrite your best email in a different tone (more urgent / more formal / more concise).

■ **Deliverable:** 2 complete email prompts + both emails reviewed and rated.

■ Scenario 1: General — Launch Delay Approval

CONTEXT	Product launch has critical QA findings requiring a two-week delay.
ROLE	Product manager
TASK	Request senior-management approval for the delay and communicate the revised timeline.
FORMAT	Professional email under 200 words.
CONSTRAINTS	Clearly explain the reason, request approval, and include the revised timeline.

Full prompt:

Write your answer / prompt here:

Act as an experienced product manager. Draft a professional email to senior management requesting approval for a two-week delay to the product launch due to critical QA findings. Clearly explain that the delay is necessary to address the identified issues and reduce launch risk. Include the proposed revised launch timeline and a clear request for approval. Keep the email under 200 words, use a professional and concise tone, and structure it so that the reason, impact, revised timeline, and requested decision are easy to identify.

■ Email is professional & ready to send ■ Needs minor edits ■ Needs major revision

■■■ Scenario 2: Dev — Emergency Production Access

CONTEXT	Requesting emergency access to production logs for investigation
---------	--

ROLE	Act as an experienced product manager
TASK	Draft a professional email to senior management requesting approval for a two-week delay to the product launch due to critical QA findings
FORMAT	Keep the email under 200 words,
CONSTRAINTS	use a professional and concise tone,

Full prompt:

Write your answer / prompt here:

Act as an experienced product manager. Draft a professional email to senior management requesting approval for a two-week delay to the product launch due to critical QA findings. Clearly explain that the delay is necessary to address the identified issues and reduce launch risk. Include the proposed revised launch timeline and a clear request for approval. Keep the email under 200 words, use a professional and concise tone, and structure it so that the reason, impact, revised timeline, and requested decision are easy to identify.

■ Email is professional & ready to send ■ Needs minor edits ■ Needs major revision

■ **BONUS** — Tone Variation (paste the AI's alternative version below):

Write your answer / prompt here:

Task 7 Technical Documentation Generation

■ 25 min

● Intermediate

■ Instructions:

1. Practice generating three types of technical documentation using AI.
2. DOC TYPE 1 — Docstring: Write a prompt asking AI to add a Google-style docstring to the sample Python function provided below.
3. DOC TYPE 2 — README Section: Write a prompt to generate an 'Installation & Setup' README section for a Node.js REST API. Include: prerequisites, clone, install dependencies, configure .env, run dev server.
4. DOC TYPE 3 — Commit Message: Write a prompt asking AI to generate a conventional commit message for these changes: 'Added Redis caching layer to the /products endpoint, reducing average response time from 480ms to 35ms. Also fixed a null pointer bug when product category is undefined.'
5. Test all three prompts. Note which output required the least editing.

■ **Deliverable:** 3 documentation prompts written + one 'least editing needed' observation.

```
# Sample function for documentation exercise:
def process_user_order(user_id, items, discount_code=None):
    total = sum(item['price'] * item['qty'] for item in items)
    if discount_code:
        discount = get_discount(discount_code)
        total = total * (1 - discount)
    if total < 0:
        raise ValueError('Total cannot be negative')
    return {'user_id': user_id, 'total': round(total, 2), 'items_count': len(items)}
```

Doc Type 1 — Docstring Prompt:

Write your answer / prompt here:

Act as a senior Python developer. Add a Google-style docstring to the following function:

```
def process_user_order(user_id, items, discount_code=None):
    total = sum(item['price'] * item['qty'] for item in items)
    if discount_code:
        discount = get_discount(discount_code)
        total = total * (1 - discount)
    if total < 0:
        raise ValueError('Total cannot be negative')
    return {'user_id': user_id, 'total': round(total, 2), 'items_count': len(items)}
```

Document the function's purpose, Args, Returns, and Raises. Do not change the function's implementation. Keep the docstring concise and follow standard Google Python documentation conventions.

Doc Type 2 — README Section Prompt:

Write your answer / prompt here:

Act as a technical writer and Node.js backend developer. Write an "Installation & Setup" section for a README for a Node.js REST API.

Include these subsections:

1. Prerequisites
2. Clone the repository
3. Install dependencies
4. Configure the .env file
5. Run the development server

Use Markdown formatting. Include realistic command examples using placeholders where repository-specific information is unknown. Do not invent project-specific environment variables or URLs.

Doc Type 3 — Commit Message Prompt:

Write your answer / prompt here:

Act as a developer familiar with Conventional Commits. Generate an appropriate Conventional Commit message for these changes:

- Added a Redis caching layer to the /products endpoint, reducing average response time from 480ms to 35ms.
- Fixed a null pointer bug when the product category is undefined.

Return only the commit type, optional scope, and concise commit description. Follow Conventional Commits syntax.

Which doc type required the least editing? ■ Docstring ■ README ■ Commit Message

Why? Commit message. The commit-message task has a narrowly defined output format and a small amount of source information, so the AI has fewer opportunities to introduce unnecessary content.

Task 8 Meeting Notes & Report Generation

■ 15 min

● Intermediate

■ Instructions:

1. Read the raw meeting notes below.
2. Write a prompt asking AI to organize these notes into a professional meeting summary with sections: Attendees, Key Decisions, Action Items (with owner and deadline), and Blockers.
3. Write a second prompt asking AI to convert the same notes into a 5-slide presentation outline.
4. Compare both outputs and note which format is most useful for different audiences.

■ Deliverable: 2 prompts written + audience suitability comparison noted.

MEETING NOTES — Sprint Planning — March 4 Present: Ahmed (backend), Sara (frontend), Khalid (DevOps), Nour (PM) Ahmed - auth service refactor done, PR open, needs review by EOD tomorrow Sara - dashboard blocked waiting for API spec from Ahmed Khalid - deployment pipeline broken on staging, no ETA yet, investigating Agreed - launch date pushed from March 10 to March 17 because of pipeline issue Nour - needs to update client on delay - email by tomorrow morning Ahmed to review Sara's frontend PR before end of week Khalid to give update on pipeline at daily standup Friday Next sprint planning March 11 at 10am

■ *Prompt 1 — Meeting Summary:*
Write your answer / prompt here:

Act as a professional project coordinator. Convert the following raw Sprint Planning meeting notes into a clear, professional meeting summary.

Use exactly these sections:

1. Attendees
2. Key Decisions
3. Action Items — include Action, Owner, and Deadline
4. Blockers

Preserve all information from the notes and do not invent missing deadlines or owners. Clearly identify information that is not specified. Use concise professional language.

Meeting notes:

[PASTE THE MARCH 4 SPRINT PLANNING NOTES HERE]

■ *Prompt 2 — Presentation Outline:*
Write your answer / prompt here:

Act as a project manager preparing a concise stakeholder presentation. Convert the following Sprint Planning meeting notes into a five-slide presentation outline.

For each slide provide:

- Slide title
- 3–5 concise bullet points
- Key message

Cover the current sprint status, key decisions, blockers, action items, and next steps. Keep the content suitable for a professional audience and do not invent information that is not present in the notes.

Meeting notes:

[PASTE THE MARCH 4 SPRINT PLANNING NOTES HERE]

Meeting summary is more useful for: ■ Developers ■ Management ■ All Presentation outline is more useful for:
■ Developers ■ Management ■ Client meetings

SECTION D | AI for Data Analysis & Code Tasks ■ 2:30 – 3:20

Tasks 9–10 focus on two of the highest-value AI use cases for developers: data and document analysis, and code-related tasks including review, debugging, and test generation.

Task 9 Document Analysis & Data Extraction

■ 25 min

● Intermediate

■ Instructions:

1. Use the sample sprint report below for all prompts in this task.
2. PROMPT 1: Write a prompt to summarize the report in exactly 5 bullet points (max 15 words each).
3. PROMPT 2: Write a prompt to extract all action items as a table with columns: Action | Owner | Deadline | Priority.
4. PROMPT 3: Write a prompt to convert the 'Performance Metrics' section into a structured JSON object.
5. PROMPT 4 (Advanced): Write a prompt asking AI to identify risks mentioned in the report and suggest mitigations.
6. Test all 4 prompts. Rate each for output quality and immediate usability.

■ **Deliverable:** 4 prompts written + quality ratings + JSON output from Prompt 3 pasted.

SPRINT 14 PERFORMANCE REPORT — Week of March 3 COMPLETED: - User authentication service refactored (owner: Ahmed, 3 days ahead of schedule) - Payment gateway integration tested - all 47 test cases passing - Mobile app API endpoints v2 deployed to staging IN PROGRESS: - Customer dashboard frontend (Sara, 60% complete, blocked on API spec) - CI/CD pipeline fix for staging environment (Khalid, ETA: March 7) BLOCKERS: - Staging deployment broken since March 2 - no prod deployments possible until resolved - API spec for dashboard not finalized - Sara blocked for 2+ days PERFORMANCE METRICS: - Sprint velocity: 42 story points (target: 45, -6.7%) - Bug count opened this sprint: 7 (4 resolved, 3 open) - Code review turnaround: avg 18 hours (target: 24 hours) ■ - Test coverage: 74% (target: 80%, gap: -6%) ACTION ITEMS: - Ahmed: Share API spec with Sara by March 5 - Khalid: Resolve pipeline issue by March 7 - Nour: Notify client of revised launch date by March 5 - All: Increase test coverage to 80% before next sprint review

Prompt 1 — 5-Bullet Summary:

Write your answer / prompt here:

Act as a project manager analyzing the Sprint 14 Performance Report below. Summarize the entire report in exactly 5 bullet points.

Requirements:

- Each bullet must contain no more than 15 words.
- Cover the most important completed work, work in progress, blockers, performance metrics, and action items.
- Use only information contained in the report.
- Do not add assumptions or recommendations.

Report:

[PASTE THE SPRINT 14 PERFORMANCE REPORT HERE]

■ Ready to use ■ Needs minor edits ■ Needs major revision

Prompt 2 — Action Items Table:

Write your answer / prompt here:

Extract every action item from the Sprint 14 Performance Report and present them in a Markdown table with exactly these columns:

| Action | Owner | Deadline | Priority |

Preserve the wording and deadlines from the report where possible. Assign Priority as High, Medium, or Low based only on the urgency or impact explicitly indicated in the report. If priority is not directly stated or cannot reasonably be inferred, write "Not specified." Do not invent missing information.

■ Ready to use ■ Needs minor edits ■ Needs major revision

Prompt 3 — JSON Conversion:

Write your answer / prompt here:

Extract the "Performance Metrics" section from the Sprint 14 Performance Report and convert it into a valid JSON object.

Use this structure:

```
{
  "sprint_velocity": {
    "actual": 42,
    "target": 45,
    "variance_percent": -6.7
  },
  "bug_count": {
    "opened": 7,
    "resolved": 4,
    "open": 3
  },
  "code_review_turnaround_hours": {
    "average": 18,
    "target": 24
  },
  "test_coverage_percent": {
    "actual": 74,
    "target": 80,
    "gap_percent": -6
  }
}
```

Return valid JSON only. Do not add commentary or fields that are not supported by the report.

■ Ready to use ■ Needs minor edits ■ Needs major revision

Prompt 4 — Risk Analysis:

Write your answer / prompt here:

Act as a senior project manager reviewing the Sprint 14 Performance Report. Identify every significant risk or blocker explicitly mentioned in the report.

For each risk, provide:

1. Risk
2. Evidence from the report
3. Potential impact
4. Recommended mitigation
5. Responsible owner, if specified

Distinguish clearly between risks stated in the report and mitigation recommendations generated by you. Do not invent facts about the project.

■ Ready to use ■ Needs minor edits ■ Needs major revision

■ *JSON output from Prompt 3 (paste or describe it):*

Task 10

AI-Assisted Code Review, Debugging & Test Generation

■ 25 min

● Advanced

■ Instructions:

1. Complete the 3 code-related sub-tasks below using the sample code provided.
2. SUB-TASK A — Code Review: Write a CRTFC prompt asking AI to review the Python function below for: correctness, edge cases, security, and PEP8 compliance. Then test it.
3. SUB-TASK B — Bug Investigation: The function below has a logic bug. Write a prompt asking AI to identify the bug and provide a fixed version with an explanation.
4. SUB-TASK C — Test Generation: Write a prompt asking AI to generate pytest unit tests for the fixed version of the function, covering: happy path, empty input, null values, and boundary conditions.
5. REFLECTION: Which sub-task produced the most impressive AI output? Note what surprised you.

■ Deliverable: 3 CRTFC prompts + all results noted + reflection written.

```
# Code for review - find the bug and write tests: def calculate_discount(cart_items,
promo_code): """Apply discount to cart total.""" PROMO_CODES = { 'SAVE10': 0.10,
'SAVE20': 0.20, 'VIP50': 0.50 } total = 0 for item in cart_items: total += item['price']
* item['quantity'] if promo code in PROMO CODES: discount = PROMO CODES[promo code] total
= total - discount # BUG IS HERE if total < 0: total = 0 return total
```

Sub-Task A — Code Review Prompt:

Write your answer / prompt here:

Act as a senior Python backend engineer and code reviewer. Review the following function for correctness, edge cases, security, and PEP 8 compliance.

Identify:

- Logic errors
- Incorrect calculations
- Input-validation issues
- Empty or null input handling
- Unexpected promo codes
- Security or reliability concerns
- PEP 8 violations
- Recommended improvements

For each issue, explain why it matters and provide a specific recommendation. Do not assume requirements that are not stated.

Code:

```
def calculate_discount(cart_items, promo_code):
    """Apply discount to cart total."""
    PROMO_CODES = {
        'SAVE10': 0.10,
        'SAVE20': 0.20,
        'VIP50': 0.50
    }
    total = 0
    for item in cart_items:
```

```

total += item['price'] * item['quantity']
if promo_code in PROMO_CODES:
    discount = PROMO_CODES[promo_code]
    total = total - discount
if total < 0:
    total = 0
return total

```

What did AI find / produce?

Write your answer / prompt here:

The major logic error is that the code subtracts the **discount rate itself** from the total instead of calculating the monetary discount. For example, a 10% discount should subtract $\text{total} * 0.10$, not 0.10.

Other potential issues include missing validation for cart items, missing price/quantity fields, invalid values, and handling of None or unexpected input.

Sub-Task B — Bug Investigation Prompt:

Write your answer / prompt here:

Act as a senior Python developer investigating a financial-calculation bug. Analyze the following function and identify the exact logic error.

Explain:

1. What the code currently does.
2. Why the calculation is incorrect.
3. A corrected implementation.
4. A simple numerical example demonstrating the difference between the buggy and corrected behavior.

Then provide the complete corrected function.

Do not introduce unrelated architectural changes.

Code:

```

def calculate_discount(cart_items, promo_code):
    """Apply discount to cart total."""
    PROMO_CODES = {
        'SAVE10': 0.10,
        'SAVE20': 0.20,
        'VIP50': 0.50
    }
    total = 0
    for item in cart_items:
        total += item['price'] * item['quantity']
    if promo_code in PROMO_CODES:
        discount = PROMO_CODES[promo_code]
        total = total - discount
    if total < 0:
        total = 0
    return total

```

What did AI find / produce?

Write your answer / prompt here:

The bug is:

`total = total - discount`

Here, discount represents a percentage expressed as a decimal, such as 0.10. The code therefore subtracts **0.10 currency units**, rather than 10% of the total.

The corrected calculation is:

`total = total - (total * discount)`

For example, if the cart total is 100 and the promo is SAVE10, the correct result is 90, not 99.90.

Sub-Task C — Test Generation Prompt:

Write your answer / prompt here:

Act as a senior Python QA engineer. Generate pytest unit tests for the corrected `calculate_discount()` function.

The tests must cover:

1. Happy path with no discount.
2. A valid 10% discount.
3. A valid 20% discount.
4. An empty cart.
5. Missing or null values where appropriate.
6. Invalid or unknown promo codes.
7. A boundary case such as a zero-price item or zero quantity.
8. A case where the discount would otherwise cause the total to become negative.

Use clear pytest test names and keep each test focused on one behavior. Where the expected behavior is ambiguous from the function specification, identify the assumption rather than silently inventing a requirement.

What did AI find / produce?

Write your answer / prompt here:

The tests should verify that:

- Normal cart totals are calculated correctly.
- Percentage discounts are applied correctly.
- Unknown promo codes do not modify the total.
- Empty carts are handled consistently.
- Invalid inputs are either rejected appropriately or handled according to the defined requirements.
- Boundary values such as zero quantities do not produce unexpected results.
- The final total does not become negative.

■ **Reflection — Which sub-task surprised you most and why?**

Write your answer / prompt here:

Sub-Task B surprised me most because AI quickly identified a subtle financial calculation error that could easily be overlooked during a normal code review. It also demonstrated the problem with a simple numerical example, making the bug much easier to understand.

SECTION E | Automation Templates & Personal AI Workflow ■ 3:20 – 4:00

Tasks 11–12 turn everything you practiced today into a reusable personal system. The templates you build here will save you hours every week.

Task 11 Build 3 Personal Reusable Prompt Templates

■ 25 min

● Intermediate

■ Instructions:

1. Build ONE reusable prompt template for each of the 3 categories below.
2. Use [PLACEHOLDERS] for the parts that change each time. Include all 5 CRTFC elements.
3. TEMPLATE 1 (General): A recurring communication task from your work (weekly update, client email, meeting request, etc.)
4. TEMPLATE 2 (Dev): A recurring development task (code review, PR description, bug report, test generation, documentation)
5. TEMPLATE 3 (Your choice): Any other task you do regularly that AI could help with
6. For each template: write it, test it by filling in the placeholders, then refine it once.

■ **Deliverable:** 3 complete reusable prompt templates with [PLACEHOLDERS].

■ General — Template 1: General Communication Template

Task this template is for: _____

Context: [] Role: Act as a [] Task: [] Format: [] Constraints: [] Full prompt (combine above):

After testing: what 1 thing did you refine? _____

■■■ Dev — Template 2: Developer Workflow Template

Task this template is for: _____

Context: [] Role: Act as a [] Task: [] Format: [] Constraints: [] Full prompt (combine above):

After testing: what 1 thing did you refine? _____

■ Custom — Template 3: My Choice Template

Task this template is for: _____

Context: [] Role: Act as a [] Task: [] Format: [] Constraints: [] Full prompt (combine above):

After testing: what 1 thing did you refine? _____

Task 12 Design Your Personal AI Developer Workflow

15 min

● Advanced

■ Instructions:

1. List your top 8 most repetitive or time-consuming development and communication tasks.
2. For each, complete the row in the workflow table: tool, prompt type, and estimated time saved per week.
3. Circle your top 3 highest-impact tasks.
4. Prepare to present ONE workflow example to the group in 60 seconds — explain: what the task is, which AI tool you will use, and how much time you expect to save.

■ Deliverable: Completed workflow table + 1 example prepared for group sharing.

#	Task Description	Type	AI Tool	Template Name	Est. Time Saved/Week
1	Fixing Java bugs and errors	■ Dev	Copilot	Java bug fixing	1.5 hours
2	Reviewing Java/Spring Boot code	■ Dev	Claude	Java code review	2 hours
3	Writing unit tests	■ Dev	Claude	Test generation	2 hours
4	Writing professional emails	■ General	ChatGPT	Professional email	20 mins
5	Comparing products before buying	■ General	ChatGPT	Product comparison	30 mins
6	Learning new topics	■ Dev	Gemini	Learning helper	1 hour
7	Reviewing code	■ Dev	Claude	Code review	1.5 hours
8		■ General ■ Dev			

■ My 60-Second Workflow Pitch (prepare here):

The task I chose: /finding and fixing code error

AI tool I will use: Claude

The prompt template (summarize): I give AI my code and explain the problem. AI finds the error, explains it, and gives me the corrected code.

Estimated time saved: _____ Impact on my work: _____

■ Day 1 — Final Reflection

Complete before the session ends:

The most useful general AI use case I discovered today:

Using AI to write and improve emails and reports

The most useful developer AI use case I will start this week:

Using AI to find and fix code errors faster and explain why the error happens.

One thing I now understand about AI that I did not before:

I now understand that the quality of the prompt affects the quality of the AI response.

One question I still have about using AI in my development workflow:

How can I use AI in my development workflow while making sure the code is secure and correct?

My commitment: the first AI-assisted task I will do at work tomorrow:

I will use AI to review and help fix a code error, then test the solution myself before using it.
